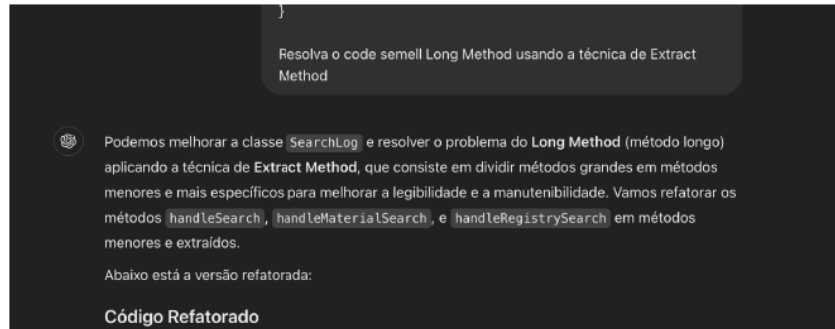


Long Method SearchLog

SearchLog

1. Foi enviado para a IA o código SearchLog com o seguinte comando:



```
package org.example.studysearch;

import org.example.studycards.CardManager;
import org.example.studyplanner.HabitTracker;
import org.example.studyplanner.TODOTracker;
import org.example.studyregistry.StudyMaterial;
import org.example.studyregistry.StudyTaskManager;

import java.util.ArrayList;
import java.util.Collections;
import java.util.HashMap;
import java.util.List;
import java.util.Map;

public class SearchLog {
    private final List<String> searchHistory;
    private final Map<String, Integer> searchCount;
    private boolean isLocked;
    private int numUsages;
    private String logName;

    public SearchLog(String logName) {
        this.searchHistory = new ArrayList<>();
        this.searchCount = new HashMap<>();
        this.logName = logName;
        this.numUsages = 0;
        this.isLocked = false;
    }

    public List<String> handleSearch(String text) {
        ensureUnlocked();
        List<String> results = gatherAllSearchResults(text);
        logSearch(text);
        appendLogNameToResults(results);
        return results;
    }

    public List<String> handleMaterialSearch(String text) {
        ensureUnlocked();
        List<String> results = gatherMaterialSearchResults(text);
        logSearch(text);
        appendLogNameToResults(results);
        return results;
    }
}
```

```

public List<String> handleRegistrySearch(String text) {
    ensureUnlocked();
    List<String> results = gatherRegistrySearchResults(text);
    logSearch(text);
    appendLogNameToResults(results);
    return results;
}

private void ensureUnlocked() {
    if (isLocked) {
        throw new IllegalStateException("SearchLog is locked. Cannot perform a search.");
    }
}

private List<String> gatherAllSearchResults(String text) {
    List<String> results = new ArrayList<>();
    results.addAll(CardManager.getCardManager().searchInCards(text));
    results.addAll(HabitTracker.getHabitTracker().searchInHabits(text));
    results.addAll(TodoTracker.getInstance().searchInTodos(text));
    results.addAll(StudyMaterial.getStudyMaterial().searchInMaterials(text));
    results.addAll(StudyTaskManager.getStudyTaskManager().searchInRegistries(text));
    return results;
}

private List<String> gatherMaterialSearchResults(String text) {
    List<String> results = new ArrayList<>();
    results.addAll(StudyMaterial.getStudyMaterial().searchInMaterials(text));
    return results;
}

private List<String> gatherRegistrySearchResults(String text) {
    List<String> results = new ArrayList<>();
    results.addAll(CardManager.getCardManager().searchInCards(text));
    results.addAll(HabitTracker.getHabitTracker().searchInHabits(text));
    results.addAll(TodoTracker.getInstance().searchInTodos(text));
    results.addAll(StudyTaskManager.getStudyTaskManager().searchInRegistries(text));
    return results;
}

private void appendLogNameToResults(List<String> results) {
    results.add("\nLogged in: " + this.logName);
}

public void logSearch(String term) {
    ensureUnlocked();
    searchHistory.add(term);
    searchCount.put(term, searchCount.getOrDefault(term, 0) + 1);
    numUsages++;
}

public List<String> getSearchHistory() {
    return Collections.unmodifiableList(searchHistory);
}

public int getSearchCount(String term) {
    return searchCount.getOrDefault(term, 0);
}

public List<String> getTopSearches() {
    int maxCount = searchCount.values().stream().max(Integer::compareTo).orElse(0);
    List<String> topSearches = new ArrayList<>();
    for (Map.Entry<String, Integer> entry : searchCount.entrySet()) {
        if (entry.getValue() == maxCount) {

```

```

        topSearches.add(entry.getKey());
    }
}
return topSearches;
}

public void lock() {
    isLocked = true;
}

public void unlock() {
    isLocked = false;
}

public int getNumUsages() {
    return numUsages;
}

public String getLogName() {
    return logName;
}

public void setLogName(String logName) {
    if (logName == null || logName.isBlank()) {
        throw new IllegalArgumentException("Log name cannot be null or blank.");
    }
    this.logName = logName;
}

public void addSearchHistory(String query) {
    searchHistory.add(query);
}

public void setNumUsages(int numUsages) {
    this.numUsages = numUsages;
}
}

```

Principais mudanças

1. Extração de Métodos:

- `gatherAllSearchResults`: Coleta resultados de todas as fontes.
- `gatherMaterialSearchResults`: Coleta apenas resultados de materiais.
- `gatherRegistrySearchResults`: Coleta resultados de registros.
- `appendLogNameToResults`: Adiciona o nome do log aos resultados.
- `ensureUnlocked`: Verifica se o log está desbloqueado antes de qualquer operação.

2. Melhoria da Legibilidade:

- Métodos pequenos e focados em uma única responsabilidade tornam o código mais claro.

3. Reutilização de Código:

- Reduziu-se duplicação ao reutilizar métodos como `logSearch` e `appendLogNameToResults`.

Essa abordagem torna o código mais modular, fácil de testar e estender.

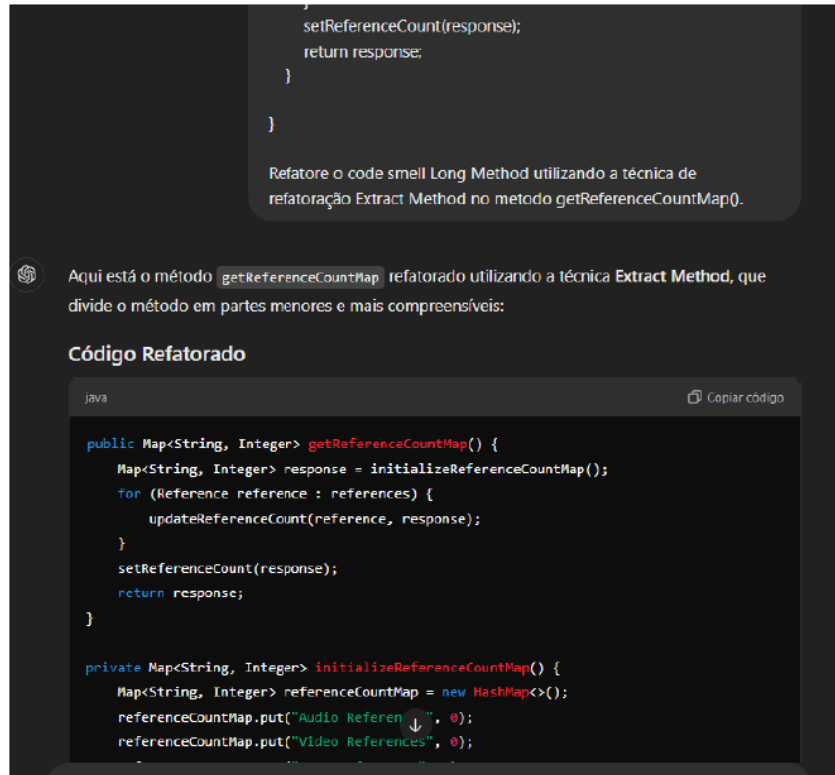
🔊 📄 📌 🔒 ↺

2. Código funcional e sem code smells

Long Method StudyMaterial

StudyMaterial

1. Foi enviado para a IA o código StudyMaterial com o seguinte comando:



//REFATORAÇÃO DA IA

```
public Map<String, Integer> getReferenceCountMap() {
    Map<String, Integer> response = initializeReferenceCountMap();
    for (Reference reference : references) {
        updateReferenceCount(reference, response);
    }
    setReferenceCount(response);
    return response;
}

private Map<String, Integer> initializeReferenceCountMap() {
    Map<String, Integer> referenceCountMap = new HashMap<>();
    referenceCountMap.put("Audio References", 0);
    referenceCountMap.put("Video References", 0);
    referenceCountMap.put("Text References", 0);
    return referenceCountMap;
}

private void updateReferenceCount(Reference reference, Map<String, Integer> referenceCountMap) {
    if (reference.getClass() == AudioReference.class) {
        incrementCount("Audio References", referenceCountMap);
    } else if (reference.getClass() == VideoReference.class) {
        handleVideoReference(reference, referenceCountMap);
    } else if (reference.getClass() == TextReference.class) {
        handleTextReference(reference, referenceCountMap);
    }
}
```

```

private void handleVideoReference(Reference reference, Map<String, Integer> referenceCountMap) {
    if (((VideoReference) reference).handleStreamAvailability()) {
        incrementCount("Video References", referenceCountMap);
    }
}

private void handleTextReference(Reference reference, Map<String, Integer> referenceCountMap) {
    if (((TextReference) reference).handleTextAccess()) {
        incrementCount("Text References", referenceCountMap);
    }
}

private void incrementCount(String key, Map<String, Integer> referenceCountMap) {
    int currentCount = referenceCountMap.get(key);
    referenceCountMap.put(key, currentCount + 1);
}

```

Explicação da Refatoração:

1. `initializeReferenceCountMap`:
 - Criado para inicializar o mapa com valores padrão.
2. `updateReferenceCount`:
 - Trata o incremento dos contadores para diferentes tipos de referências.
3. `handleVideoReference`:
 - Isola a lógica específica para lidar com referências de vídeo.
4. `handleTextReference`:
 - Isola a lógica específica para lidar com referências de texto.
5. `incrementCount`:
 - Isola a lógica de incremento para reduzir duplicação.

Benefícios:

- **Legibilidade:** Cada método tem uma única responsabilidade.
- **Reutilização:** Métodos como `incrementCount` podem ser reutilizados.
- **Manutenibilidade:** Adicionar ou alterar tipos de referências agora é mais fácil, pois a lógica está segmentada.

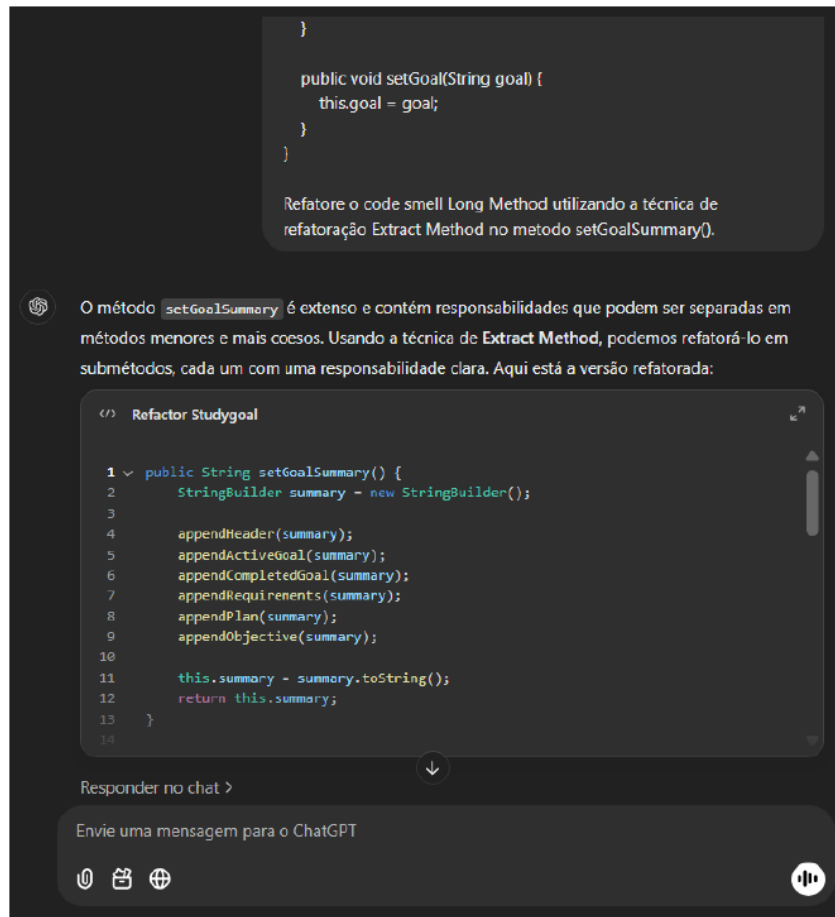
🔊 📄 👍 🗑️ ↺

2. Código funcional e sem code smells

Long Method StudyGoal

StudyGoal

1. Foi enviado para a IA o código StudyGoal com o seguinte comando:



//REFATORAÇÃO DA IA

```
public String setGoalSummary() {
    StringBuilder summary = new StringBuilder();

    appendHeader(summary);
    appendActiveGoal(summary);
    appendCompletedGoal(summary);
    appendRequirements(summary);
    appendPlan(summary);
    appendObjective(summary);

    this.summary = summary.toString();
    return this.summary;
}

private void appendHeader(StringBuilder summary) {
    summary.append("Goal Summary:\n\n");
}

private void appendActiveGoal(StringBuilder summary) {
    if (this.isActive) {
        summary.append("Active Goal:\n").append(goal).append("\n\n");
    }
}
```

```

}

private void appendCompletedGoal(StringBuilder summary) {
    if (this.isCompleted) {
        summary.append("Completed Goal:\n").append(goal).append("\n\n");
    }
}

private void appendRequirements(StringBuilder summary) {
    if (this.goalRequirements != null && !this.goalRequirements.isEmpty()) {
        summary.append("Requirements:\n");
        for (String requirement : this.goalRequirements) {
            summary.append(requirement).append(", ");
        }
        // Remove trailing comma and space if present
        if (summary.length() > 0) {
            summary.setLength(summary.length() - 2);
        }
        summary.append("\n\n");
    }
}

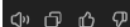
private void appendPlan(StringBuilder summary) {
    if (this.studyPlan != null) {
        summary.append("Plan:\n").append(this.studyPlan.toString()).append("\n\n");
    }
}

private void appendObjective(StringBuilder summary) {
    if (this.studyObjective != null) {
        summary.append("Objective:\n").append(this.studyObjective.toString()).append("\n\n");
    }
}
}

```

Benefícios da Refatoração

1. **Clareza:** Cada método tem uma responsabilidade específica, facilitando a leitura e manutenção do código.
2. **Reutilização:** Os submétodos podem ser reutilizados em outras partes do código, se necessário.
3. **Testabilidade:** Métodos menores são mais fáceis de testar isoladamente.
4. **Redução da Complexidade:** O método principal (`setGoalSummary`) se torna mais simples e com responsabilidade unificada.

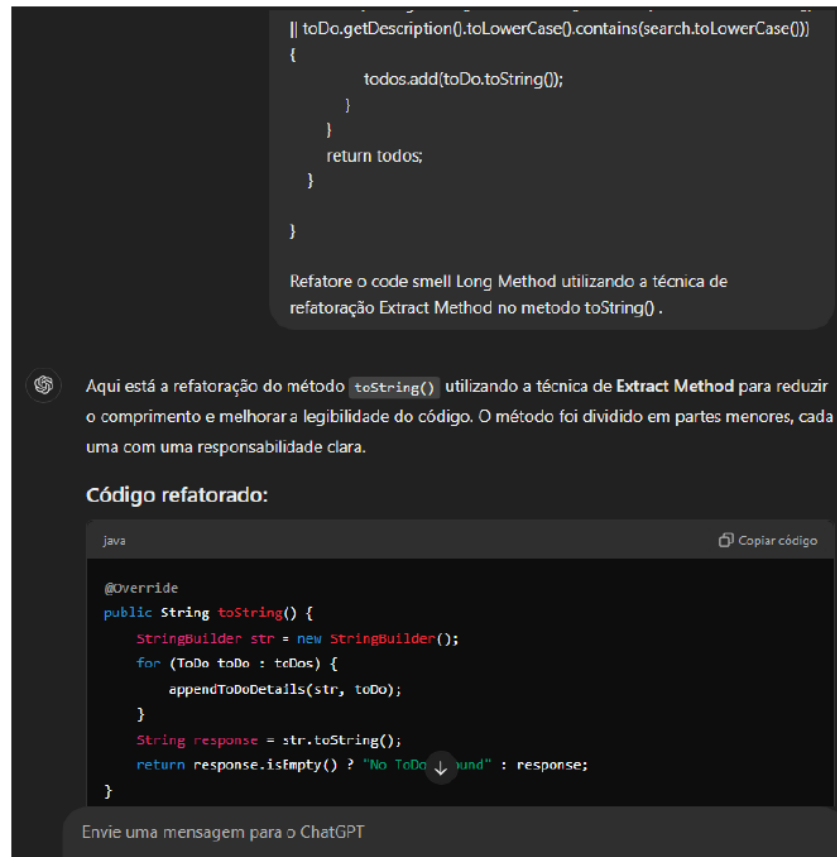


3. Código funcional e sem code smells

Long Method ToDoTracker

ToDoTracker

1. Foi enviado para a IA o código ToDoTracker com o seguinte comando:



```
//REFATORAÇÃO DA IA

@Override
public String toString() {
    StringBuilder str = new StringBuilder();
    for (ToDo toDo : todos) {
        appendToDoDetails(str, toDo);
    }
    String response = str.toString();
    return response.isEmpty() ? "No ToDos found" : response;
}

private void appendToDoDetails(StringBuilder str, ToDo toDo) {
    str.append(toDo.toString()).append("\n");
    appendToDoExecutionTimes(str, toDo.getId());
}

private void appendToDoExecutionTimes(StringBuilder str, Integer id) {
    List<LocalDateTime> todosDate = this.tracker.get(id);
    if (todosDate == null) {
        str.append("No tracks found\n");
    } else {
        for (LocalDateTime ldt : todosDate) {
            str.append(formatDate(ldt)).append("\n");
        }
    }
}
```



```

}

private String formatDate(LocalDateTime dateTime) {
    String pattern = "yyyy-MM-dd HH:mm:ss";
    DateTimeFormatter formatter = DateTimeFormatter.ofPattern(pattern);
    return formatter.format(dateTime);
}

```

Alterações realizadas:

1. `appendToDoDetails`: Método extraído para adicionar os detalhes básicos de um `ToDo` ao `StringBuilder`. Ele também chama `appendToDoExecutionTimes` para incluir os tempos de execução.
2. `appendToDoExecutionTimes`: Método extraído para tratar a inclusão dos horários de execução no `StringBuilder`. Lida com o caso de horários inexistentes e adiciona os horários formatados.
3. `formatDate`: Método extraído para centralizar a lógica de formatação de datas, garantindo maior reutilização e clareza.

Benefícios da refatoração:

- **Redução da complexidade:** O método `toString()` agora é mais curto e legível.
- **Responsabilidade única:** Cada método trata uma parte específica do processamento.
- **Reutilização:** A lógica de formatação de datas está encapsulada em um método separado, permitindo reutilização em outros locais.

Essa refatoração melhora a manutenibilidade e torna o código mais fácil de entender.

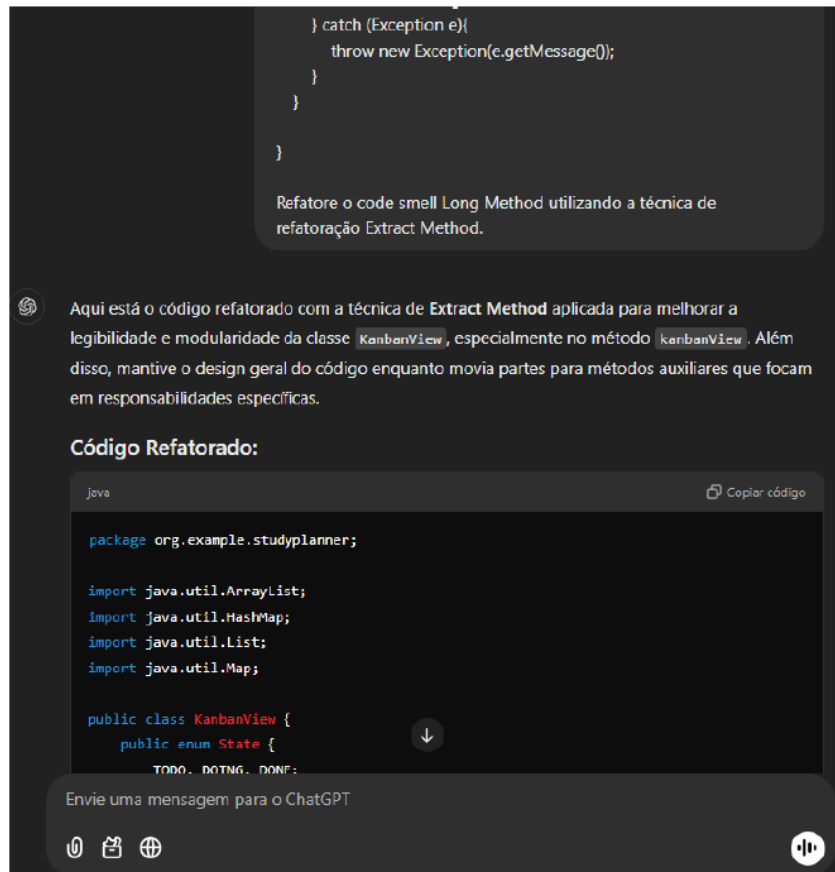
< 2/2 > 🔊 📄 🌟 🔄

3. Código funcional e sem code smells

Long Method KanbanView

KanbanView

1. Foi enviado para a IA o código KanbanView com o seguinte comando:



```
//REFATORAÇÃO DA IA

package org.example.studyplanner;

import java.util.ArrayList;
import java.util.HashMap;
import java.util.List;
import java.util.Map;

public class KanbanView {
    public enum State {
        TODO, DOING, DONE;
    }

    HabitTracker habitTracker = null;
    TodoTracker todoTracker = null;
    Map<State, List<PlannerMaterial>> kanban = null;

    public KanbanView(HabitTracker habitTracker, TodoTracker todoTracker) {
        this.habitTracker = habitTracker;
        this.todoTracker = todoTracker;
        this.kanban = new HashMap<>();
        this.kanban.put(State.TODO, new ArrayList<>());
        this.kanban.put(State.DOING, new ArrayList<>());
        this.kanban.put(State.DONE, new ArrayList<>());
    }
}
```

```

    }

    public List<PlannerMaterial> getKanbanByState(State state) {
        return kanban.get(state);
    }

    public void addHabitToKanban(State state, Integer id) throws Exception {
        try {
            Habit toAdd = this.habitTracker.getHabitById(id);
            if (toAdd == null) {
                throw new Exception("Habit not found with id: " + id);
            }
            kanban.get(state).add(toAdd);
        } catch (Exception e) {
            throw new Exception(e.getMessage());
        }
    }

    public void addToDoToKanban(State state, Integer id) throws Exception {
        try {
            ToDo toAdd = this.todoTracker.getToDoById(id);
            if (toAdd == null) {
                throw new Exception("ToDo not found with id: " + id);
            }
            kanban.get(state).add(toAdd);
        } catch (Exception e) {
            throw new Exception(e.getMessage());
        }
    }

    public void removeHabitFromKanban(State state, Integer id) throws Exception {
        try {
            Habit toRemove = this.habitTracker.getHabitById(id);
            if (toRemove == null) {
                throw new Exception("No habit found with id: " + id);
            }
            kanban.get(state).remove(toRemove);
        } catch (Exception e) {
            throw new Exception(e.getMessage());
        }
    }

    public void removeToDoFromKanban(State state, Integer id) throws Exception {
        try {
            ToDo toRemove = this.todoTracker.getToDoById(id);
            if (toRemove == null) {
                throw new Exception("No todo found with id: " + id);
            }
            kanban.get(state).remove(toRemove);
        } catch (Exception e) {
            throw new Exception(e.getMessage());
        }
    }

    public String kanbanView() throws Exception {
        try {
            if (kanban.isEmpty()) {
                throw new Exception("No material found");
            }
            StringBuilder sb = new StringBuilder();
            sb.append("[ Material ToDo: ");
            sb.append(System.lineSeparator());
            appendMaterialsByState(sb, State.TODO, "No material found");
        }
    }

```

```

        sb.append(System.lineSeparator());
        sb.append("Material in progress:");
        sb.append(System.lineSeparator());
        appendMaterialsByState(sb, State.DOING, "No material found");

        sb.append(System.lineSeparator());
        sb.append("Material completed:");
        sb.append(System.lineSeparator());
        appendMaterialsByState(sb, State.DONE, "No material found");
        sb.append("]");

        return sb.toString();
    } catch (Exception e) {
        throw new Exception(e.getMessage());
    }
}

// Método auxiliar para refatoração
private void appendMaterialsByState(StringBuilder sb, State state, String emptyMessage) {
    List<PlannerMaterial> materials = kanban.get(state);
    if (materials == null || materials.isEmpty()) {
        sb.append(emptyMessage);
    } else {
        for (PlannerMaterial material : materials) {
            sb.append(", ").append(material.toString());
        }
    }
}
}
}

```

Alterações Principais:

1. `appendMaterialsByState`:

- Extraí a lógica de adicionar os materiais no `StringBuilder` para um método auxiliar, tornando o código mais modular e legível.
- Este método recebe o estado (`State`), o `StringBuilder`, e a mensagem padrão para o caso de lista vazia.

2. Leitura Melhorada:

- A responsabilidade de cada trecho no método `kanbanView` ficou mais clara. Agora, ele foca na estrutura geral da saída e delega a criação do conteúdo ao método auxiliar.

3. Manutenção Simplificada:

- Adicionar ou alterar a lógica para exibir os estados específicos agora exige mudanças apenas no método auxiliar, reduzindo duplicação.

Com essas alterações, o código é mais fácil de manter e entender.

2. Code smell continuou, então o seguinte comando foi mandado:

continua o code smell, é aqui ó:

```
public String kanbanView() throws Exception {
    try {
        if (kanban.isEmpty()) {
            throw new Exception("No material found");
        }
        StringBuilder sb = new StringBuilder();
        sb.append("[ Material ToDo: ");
        sb.append(System.lineSeparator());
        appendMaterialsByState(sb, State.TODO, "No material found");

        sb.append(System.lineSeparator());
        sb.append("Material in progress:");
        sb.append(System.lineSeparator());
        appendMaterialsByState(sb, State.DOING, "No material found");

        sb.append(System.lineSeparator());
        sb.append("Material completed:");
        sb.append(System.lineSeparator());
        appendMaterialsByState(sb, State.DONE, "No material found");
        sb.append("]");

        return sb.toString();
    } catch (Exception e) {
        throw new Exception(e.getMessage());
    }
}
```



O método `kanbanView` ainda tem um **code smell** relacionado ao **Long Method**, mesmo com o uso do método auxiliar `appendMaterialsByState`. Vamos continuar a refatoração utilizando **Extract Method** para melhorar ainda mais a clareza e reduzir a responsabilidade desse método.

Refatoração Continuada

Para tornar o código mais modular, vou dividir a lógica do método `kanbanView` em métodos auxiliares adicionais que tratam de cada seção (TODO, DOING e DONE) separadamente. Isso deixará o método principal mais limpo.

Código Refatorado:

//REFATORAÇÃO DA IA

```
public String kanbanView() throws Exception {
    try {
        if (kanban.isEmpty()) {
            throw new Exception("No material found");
        }
        StringBuilder sb = new StringBuilder();
        sb.append(buildSection("Material ToDo", State.TODO));
        sb.append(buildSection("Material in progress", State.DOING));
        sb.append(buildSection("Material completed", State.DONE));
        return sb.toString();
    } catch (Exception e) {
        throw new Exception(e.getMessage());
    }
}

// Método auxiliar para construir cada seção do kanban
private String buildSection(String header, State state) {
    StringBuilder section = new StringBuilder();
    section.append(header).append(":");
    section.append(System.lineSeparator());
    appendMaterialsByState(section, state, "No material found");
    section.append(System.lineSeparator());
}
```

```

        return section.toString();
    }

    // Método auxiliar para adicionar materiais ao StringBuilder
    private void appendMaterialsByState(StringBuilder sb, State state, String emptyMessage) {
        List<PlannerMaterial> materials = kanban.get(state);
        if (materials == null || materials.isEmpty()) {
            sb.append(emptyMessage);
        } else {
            for (PlannerMaterial material : materials) {
                sb.append(", ").append(material.toString());
            }
        }
    }
}

```

Alterações Principais:

1. Novo Método: `buildSection`:

- Este método recebe o cabeçalho (nome da seção) e o estado correspondente (`State`) e constrói o conteúdo da seção no `StringBuilder`.
- Ele usa o método `appendMaterialsByState` para evitar duplicação de lógica.

2. `kanbanView` Reduzido:

- Agora, o método principal `kanbanView` apenas verifica se o Kanban está vazio e delega a construção de cada seção ao método `buildSection`.

3. Separação de Responsabilidades:

- Cada método tem uma única responsabilidade:
 - `kanbanView`: Estrutura geral da visualização.
 - `buildSection`: Construção de cada seção do Kanban.
 - `appendMaterialsByState`: Inserção de materiais de um estado específico.

4. Manutenção Simplificada:

- Alterações no layout ou na lógica de exibição de cada estado agora podem ser feitas no método `buildSection` ou em `appendMaterialsByState`, sem afetar o restante do código.

Resultado:

Agora o código está mais modular, legível e alinhado ao princípio **SRP (Single Responsibility Principle)**. Ele facilita a manutenção e futuras extensões da lógica de exibição do Kanban.

🔊 📄 📌 🔄 ⌵



Envie uma mensagem para o ChatGPT



3. Código funcional e sem codesmells

Long Method **LeitnerSystem**

LeitnerSystem

1. Foi enviado para a IA o código LeitnerSystem com o seguinte comando:



```
//REFATORAÇÃO DA IA  
  
package org.example.studycards;  
  
import java.util.ArrayList;  
import java.util.Arrays;  
import java.util.List;  
  
public class LeitnerSystem extends StudyMethod {  
    List<Box> boxes = null;  
  
    public LeitnerSystem(String methodName) {  
        super(methodName);  
        initializeBoxes();  
    }  
  
    private void initializeBoxes() {  
        boxes = new ArrayList<>(Arrays.asList(new Box(), new Box(), new Box(), new Box(), new Box  
()));  
    }  
  
    @Override  
    public String getMethodName() {  
        return this.methodName;  
    }  
}
```

```

@Override
void setMethodName(String methodName) {
    this.methodName = methodName;
}

@Override
public String toString() {
    return buildBoxesDescription();
}

private String buildBoxesDescription() {
    StringBuilder response = new StringBuilder();
    int index = 0;
    for (Box box : boxes) {
        response.append("Box ").append(index).append(": ").append(box.toString()).append("\n");
        index++;
    }
    return response.toString();
}

public void clearBoxes() {
    initializeBoxes();
}

public List<Box> getBoxes() {
    return boxes;
}

public String getRandomCard(List<Box> otherBoxes) {
    if (isInvalidBoxList(otherBoxes)) {
        return null;
    }
    return generateRandomCardResponse(otherBoxes);
}

private boolean isInvalidBoxList(List<Box> otherBoxes) {
    return otherBoxes == null || otherBoxes.isEmpty();
}

private String generateRandomCardResponse(List<Box> otherBoxes) {
    Box allBoxes = combineAllBoxes(otherBoxes);
    Integer randomCard = allBoxes.getRandomCard();
    return randomCard == null ? "No card found" : formatCardResponse(randomCard);
}

private Box combineAllBoxes(List<Box> otherBoxes) {
    Box allBoxes = new Box();
    for (Box box : otherBoxes) {
        allBoxes.addCards(box.getCards());
    }
    return allBoxes;
}

private String formatCardResponse(Integer randomCard) {
    CardManager manager = CardManager.getCardManager();
    Card card = manager.getCard(randomCard);
    return String.format("[%d] The random question was: %s | The answer is: %s",
        randomCard, card.getQuestion(), card.getAnswer());
}

public void addCardToBox(Integer id, Integer boxId) {
    this.boxes.get(boxId).addCard(id);
}

```



```

public void removeCardFromBox(Integer id, Integer boxId) {
    this.bboxes.get(boxId).removeCard(id);
}

public Card takeCardFromBox(Integer boxId) {
    Integer cardId = bboxes.get(boxId).getRandomCard();
    return this.cardManager.getCard(cardId);
}

public void boxIdValidation(Integer boxId) throws Exception {
    if (isInvalidBoxId(boxId)) {
        throw new Exception("Invalid box ID");
    }
}

private boolean isInvalidBoxId(Integer boxId) {
    return boxId == null || boxId > (bboxes.size() - 1) || boxId <= 0;
}

public void upgradeCard(Integer cardId, Integer boxId) throws Exception {
    boxIdValidation(boxId);
    processCardUpgrade(cardId, boxId);
}

private void processCardUpgrade(Integer cardId, Integer boxId) throws Exception {
    Box refBox = bboxes.get(boxId);
    if (refBox.hasCard(cardId)) {
        throw new Exception("No card Found");
    }
    refBox.removeCard(cardId);
    bboxes.get(Math.min(boxId + 1, 4)).addCard(cardId);
}

public void downgradeCard(Integer cardId, Integer boxId) throws Exception {
    boxIdValidation(boxId);
    processCardDowngrade(cardId, boxId);
}

private void processCardDowngrade(Integer cardId, Integer boxId) throws Exception {
    Box refBox = bboxes.get(boxId);
    if (refBox.hasCard(cardId)) {
        throw new Exception("No card Found");
    }
    refBox.removeCard(cardId);
    bboxes.get(Math.max(boxId - 1, 0)).addCard(cardId);
}
}

```

O que foi feito:

1. `toString`:
 - Extraído o método `buildBoxesDescription` para gerar a descrição das caixas.
2. `clearBoxes`:
 - Substituído o código duplicado pela chamada ao novo método `initializeBoxes`.
3. `getRandomCard`:
 - Dividido em:
 - `isInvalidBoxList` para verificar validade da lista.
 - `generateRandomCardResponse` para construir a resposta.
 - `combineAllBoxes` para combinar cartas de todas as caixas.
 - `formatCardResponse` para formatar a resposta do cartão.
4. `boxIdValidation`:
 - Extraído o método `isInvalidBoxId` para validar IDs de caixas.
5. `upgradeCard` e `downgradeCard`:
 - Extraídos os métodos `processCardUpgrade` e `processCardDowngrade` para isolar a lógica principal.

Benefícios

- Métodos menores e mais claros.
- Reutilização de código em métodos auxiliares.
- Melhor organização e legibilidade geral do código.



2. Código funcional, passou nos testes e sem code smells